

CHƯƠNG 3: THIẾT KẾ CƠ SỞ DỮ LIỆU VÀ KIẾN TRÚC CHI TIẾT

Hệ thống Chatbot Thông tin Sinh viên được triển khai trên nền PHP, MySQL và một service Python riêng cho chức năng đọc văn bản. Để các chức năng tra cứu, trả lời tự động, lưu lịch sử và hỗ trợ sinh viên hoạt động đồng bộ, mô hình dữ liệu cần được cụ thể hóa thành các bảng có ràng buộc rõ ràng; backend cần có các endpoint xử lý riêng cho từng nhóm nghiệp vụ; giao diện cần cung cấp điểm truy cập thống nhất cho cả Sinh viên và Admin.

Ở giai đoạn phân tích yêu cầu ban đầu, phạm vi dữ liệu tập trung vào chatbot trả lời các câu hỏi FAQ tĩnh. Khi xây dựng backend thực tế, nhóm chủ động mở rộng schema với các nhóm bảng học vụ và học phí để mô phỏng gần hơn môi trường dữ liệu trường học, đồng thời kiểm thử khả năng phân tách giữa dữ liệu động và kho tri thức. Nhờ đó, các thông tin cá nhân như điểm, lịch học hoặc học phí có thể được truy vấn trực tiếp từ cơ sở dữ liệu thay vì đưa vào prompt của mô hình AI. Phần mở rộng này bổ sung các nhánh xử lý dữ liệu động nhưng vẫn giữ mục tiêu cốt lõi là cung cấp thông tin cho sinh viên.

Thiết kế triển khai hiện tại tổ chức dữ liệu thành các nhóm chuyên biệt. Tài khoản được quản lý qua `users` và `roles`; hồ sơ sinh viên được lưu ở `student\_profiles`; kho tri thức gồm nguồn, bài viết và các đoạn nội dung; lịch sử hội thoại, đánh giá và câu hỏi chưa trả lời được lưu riêng; ticket hỗ trợ được tổ chức thành ticket chính và các tin nhắn trao đổi. Việc tách bảng giúp mỗi thành phần có ràng buộc phù hợp và giúp backend truy vấn đúng loại dữ liệu thay vì gom mọi thông tin vào một bảng FAQ duy nhất.

3.1. Chi tiết hóa schema triển khai thực tế (MySQL)

Schema chính sử dụng MySQL 8.0+ và InnoDB cho các bảng nghiệp vụ. File `database/do\_an\_udpm\_database\_complete.sql` tạo cơ sở dữ liệu `do\_an\_udpm`, đặt bộ ký tự `utf8mb4`, thiết lập múi giờ `+07:00` và tạo các bảng, khóa ngoại, index cùng các view cần thiết. `database\_full\_export.sql` lưu bản xuất tổng hợp của cấu trúc và dữ liệu. `fix\_users.sql` và `seed\_student.sql` bổ sung tài khoản mẫu, hồ sơ sinh viên và dữ liệu học vụ dùng trong quá trình chạy thử.

3.1.1. Kiểu dữ liệu, ràng buộc và index chi tiết từng bảng

Các bảng sử dụng số nguyên không dấu cho phần lớn khóa chính và khóa ngoại. Cách chọn này phù hợp với các quan hệ nối giữa tài khoản, hồ sơ, học phần, phiên chat và ticket. Những trường có tập trạng thái giới hạn được khai báo bằng `ENUM`, giúp dữ liệu chỉ nhận các giá trị đã được quy định. Nội dung dài dùng `TEXT` hoặc `LONGTEXT`; ngày và thời gian dùng `DATE`, `TIME` hoặc `DATETIME`; dữ liệu cấu hình và dữ liệu trích xuất có cấu trúc dùng `JSON`.

Index được đặt ở các cột thường xuyên xuất hiện trong khóa ngoại, điều kiện lọc, điều kiện sắp xếp hoặc tìm kiếm. Các bảng tri thức, thông báo, câu hỏi và tin nhắn có Full-Text Index để MySQL tìm kiếm theo nội dung. Về kiến trúc CSDL tri thức, bảng `knowledge\_chunks` đã được thiết kế sẵn trường `embedding\_json` để lưu dữ liệu vector phục vụ Semantic Search (tìm

kiểm ngữ nghĩa). Tuy nhiên, trong phiên bản PHP thuần hiện tại, hệ thống ưu tiên Full-Text Search kết hợp điểm phủ từ khóa, điểm ngữ cảnh, độ mới và mức ưu tiên nhằm duy trì tốc độ phản hồi trên hạ tầng máy chủ cục bộ. Trường `embedding\_json` được giữ lại như một cấu trúc mở, sẵn sàng cho việc nâng cấp module RAG lên Vector Search mà không cần thay đổi cấu trúc dữ liệu gốc.

Nhóm chức năng	Các bảng chính	Vai trò dữ liệu
Xác thực và tổ chức	roles, users, faculties, programs, student_profiles	Tài khoản, vai trò, khoa, chương trình đào tạo và hồ sơ sinh viên
Học vụ	academic_years, semesters, subjects, subject_prerequisites, course_sections, class_schedule_sessions, enrollments, exam_schedules, grades	Năm học, học kỳ, môn học, điều kiện, lớp học phần, lịch học, đăng ký, lịch thi và điểm
Học phí	tuition_invoices, tuition_invoice_items, tuition_payments, tuition_extension_requests	Hóa đơn, chi tiết khoản thu, thanh toán và gia hạn
Thông báo	announcements, announcement_audiences, academic_deadlines	Thông báo, đối tượng nhận và hạn chót
Kho tri thức	knowledge_sources, knowledge_articles, knowledge_question_examples, knowledge_chunks, search_synonyms, faq	Nguồn, bài tri thức, câu hỏi mẫu, đoạn nội dung, từ đồng nghĩa và FAQ
Chatbot và RAG	chat_sessions, chat_messages, rag_retrieval_logs, chat_feedback, unanswered_questions	Phiên chat, tin nhắn, log truy xuất, đánh giá và câu hỏi chưa có lời giải
Hỗ trợ	tickets, ticket_messages, ticket_attachments	Ticket, trao đổi hai chiều và tệp đính kèm
Vận hành	system_settings, uploaded_documents, audit_logs, api_usage_logs	Cấu hình, nhập tài liệu, nhật ký quản trị và nhật ký gọi API

Bảng 3.1. Phân nhóm các bảng trong schema triển khai

Nhóm tài khoản gồm `roles`, `users`, `faculties`, `programs` và `student\_profiles`. `roles.id` dùng `TINYINT UNSIGNED AUTO\_INCREMENT`; `roles.code` là `VARCHAR(30) UNIQUE`. Trong `users`, `username VARCHAR(80)` và `email VARCHAR(190)` có ràng buộc duy nhất, `password\_hash VARCHAR(255) NOT NULL` lưu mật khẩu đã băm, còn `role\_id` là khóa ngoại đến `roles`. Trạng thái tài khoản dùng `ENUM('active','inactive','locked','pending')`; hai index `idx\_users\_status` và `idx\_users\_name` phục vụ lọc tài khoản trong trang quản trị. `student\_profiles.user\_id` là khóa ngoại duy nhất đến `users`, còn `student\_code` là mã số sinh viên duy nhất. Khi tài khoản bị xóa, hồ sơ sinh viên liên quan được xóa theo; khi chương trình đào tạo bị xóa, hồ sơ vẫn giữ lại và chỉ mất liên kết thông qua `ON DELETE SET NULL`.

Nhóm học vụ gồm các bảng năm học, học kỳ, môn học, điều kiện môn học, lớp học phần, lịch học, đăng ký, lịch thi và điểm. `academic\_years` và `semesters` dùng `DATE` cho thời gian bắt đầu - kết thúc cùng các ràng buộc kiểm tra thứ tự ngày. `subjects.code` là mã duy nhất và có Full-Text Index trên mã, tên và mô tả. `subject\_prerequisites` dùng khóa chính ghép để tránh ghi trùng cùng một quan hệ điều kiện. `course\_sections` có khóa duy nhất trên `(semester\_id, section\_code)`, `enrollments` có khóa duy nhất trên `(student\_id, course\_section\_id)`, còn `class\_schedule\_sessions` kiểm tra ngày trong khoảng 1 đến 7 và giờ kết thúc lớn hơn giờ bắt đầu. `exam\_schedules` có index theo ngày và giờ thi; `grades` giới hạn điểm hệ 10 trong khoảng 0 đến 10 và điểm hệ 4 trong khoảng 0 đến 4.

Nhóm học phí dùng `DECIMAL(15,2)` cho các khoản tiền để tránh sai số của kiểu số thực. `tuition\_invoices.invoice\_number` và `tuition\_payments.transaction\_code` là các mã duy nhất. `outstanding\_amount` và `tuition\_invoice\_items.amount` là các cột sinh tự động từ công thức trong database. Các index trên `(student\_id, status)`, `due\_date` và `(payment\_status, paid\_at)` phục vụ tra cứu công nợ và đối soát giao dịch.

Nhóm thông báo gồm `announcements`, `announcement\_audiences` và `academic\_deadlines`. `announcements` có Full-Text Index trên tiêu đề, tóm tắt và nội dung; `academic\_deadlines` có Full-Text Index trên tiêu đề và mô tả. Index theo trạng thái, thời điểm công bố và hạn chót giúp hệ thống chỉ lấy những thông báo đang được công bố hoặc deadline còn hiệu lực.

Nhóm kho tri thức được tách thành nhiều lớp. `knowledge\_sources` lưu nguồn, đơn vị ban hành, số văn bản, đường dẫn và trạng thái. `knowledge\_articles` lưu tiêu đề, nhóm chủ đề, ý định, từ khóa, câu trả lời, phạm vi áp dụng, trạng thái kiểm duyệt, độ tin cậy và thời hạn hiệu lực. `knowledge\_question\_examples` lưu các cách hỏi mẫu. `knowledge\_chunks` lưu từng đoạn nội dung với `chunk\_index`, `chunk\_text`, số token, metadata và checksum; khóa duy nhất `(article\_id, chunk\_index)` bảo đảm thứ tự đoạn không bị trùng. `search\_synonyms` lưu các cách gọi tương đương. Bảng `faq` vẫn giữ các cột `topic\_group`, `tu\_khoa`, `noi\_dung` và `link\_dieu\_huong` để tương thích với dữ liệu FAQ ban đầu.

Nhóm hội thoại gồm `chat\_sessions`, `chat\_messages`, `rag\_retrieval\_logs`, `chat\_feedback` và `unanswered\_questions`. `chat\_sessions.session\_uuid` CHAR(36) UNIQUE định danh phiên chat; `chat\_messages` lưu người gửi, nội dung gốc, nội dung chuẩn hóa, intent, entities, tên model, độ trễ, điểm tin cậy và trạng thái trả lời. `rag\_retrieval\_logs` ghi điểm Full-Text, điểm từ khóa, điểm metadata, điểm freshness và điểm tổng hợp; trường `semantic\_score` có trong schema nhưng luồng hiện tại chưa gán giá trị. `chat\_feedback` có khóa duy nhất `(message\_id, user\_id)` để một người dùng không đánh giá trùng một câu trả lời. `unanswered\_questions` lưu câu hỏi chưa tìm được context phù hợp và số lần xuất hiện.

Nhóm hỗ trợ gồm `tickets`, `ticket\_messages` và `ticket\_attachments`. `tickets.ticket\_number` là mã duy nhất; `status` dùng các giá trị `open`, `in\_progress`, `waiting\_student`, `resolved`, `closed` và `cancelled`; index `(status, priority)` giúp Admin lọc các yêu cầu cần xử lý. `ticket\_messages` lưu người gửi, vai trò, nội dung và thời gian; index

`(ticket\_id, created\_at)` phục vụ hiển thị hội thoại theo thứ tự. `ticket\_attachments` liên kết với một tin nhắn và dùng `ON DELETE CASCADE`.

Bảng	Ràng buộc / Index	Mục đích triển khai
users	UNIQUE username/email; idx_users_status; idx_users_name	Định danh tài khoản và lọc người dùng
student_profiles	UNIQUE user_id; UNIQUE student_code; FK users/programs	Bảo đảm quan hệ một - một và không trùng MSSV
subjects	UNIQUE code; ft_subject_search	Tra cứu môn học theo mã, tên và mô tả
knowledge_articles	ft_knowledge_search; idx_knowledge_filter; idx_knowledge_intent; idx_knowledge_priority	Tìm kiếm, lọc kiểm duyệt, intent và ưu tiên
knowledge_chunks	UNIQUE article_id/chunk_index; ft_chunk_text	Giữ thứ tự đoạn và tìm nội dung
chat_sessions	UNIQUE session_uuid; idx_chat_user_activity	Định danh phiên và tải lịch sử
chat_messages	idx_message_session_time; ft_chat_content	Truy vấn hội thoại và tìm kiếm nội dung
chat_feedback	UNIQUE message_id/user_id	Ngăn đánh giá trùng
tickets	UNIQUE ticket_number; idx_ticket_status_priority; idx_ticket_student	Lọc ticket theo mã, trạng thái, ưu tiên và sinh viên

Bảng 3.2. Một số ràng buộc và index tiêu biểu

Schema còn tạo các view dùng chung. `system\_notifications` và `class\_schedules` cung cấp dữ liệu tương thích cho các màn hình portal. `v\_active\_verified\_knowledge` chỉ lấy bài tri thức đã xác minh, chưa bị xóa, còn hiệu lực và có nguồn đang hoạt động. `v\_unpaid\_tuition` phục vụ tra cứu công nợ; `v\_chat\_quality\_summary` tổng hợp số lượng câu trả lời, trạng thái thiếu context, điểm tin cậy và feedback theo ngày. Việc đặt điều kiện kiểm duyệt trong view giúp các truy vấn RAG dùng chung một quy tắc thay vì mỗi endpoint tự lọc theo một cách khác nhau.

3.1.2. Đối chiếu với script triển khai thực tế

`database/do\_an\_udpm\_database\_complete.sql` là script tạo schema nền tảng. Script tạo database, tạo các bảng nghiệp vụ, tạo view, tạo bảng `faq`, nhập dữ liệu FAQ ban đầu và chuyển dữ liệu sang `knowledge\_articles`, `knowledge\_question\_examples` và `knowledge\_chunks`. Cách tổ chức này cho phép giữ lại dữ liệu FAQ cũ trong khi bổ sung trạng thái kiểm duyệt và nguồn tri thức cho luồng chatbot.

`database\_full\_export.sql` là bản xuất tổng hợp dùng để sao lưu và đối chiếu. `fix\_users.sql` tạo lại tài khoản Admin và sinh viên cùng hồ sơ sinh viên; mật khẩu mẫu trong file là chuỗi băm để PHP có thể xác minh bằng `password\_verify()`. `seed\_student.sql` bổ sung môn học, năm học, học kỳ, lớp học phần, lịch học và dữ liệu đăng ký. Các script seed được thực hiện sau khi

schema đã tồn tại; thông tin tài khoản cuối cùng cần được kiểm tra lại sau khi chạy để tránh việc dữ liệu mẫu của một script ghi đè dữ liệu từ script khác.

Trong database chính, role được tạo bằng các mã `student`, `admin`, `staff` và `knowledge\_reviewer`. Một số script seed dùng `role\_id` dạng số, vì vậy việc chạy seed cần dựa trên database đã có bảng `roles` và đúng dữ liệu role. Đây là lý do nhóm kiểm tra lại tài khoản sau khi import thay vì chỉ dựa vào giá trị role_id được ghi cứng trong file seed.

[CHÈN HÌNH 3.1: Sơ đồ ánh xạ các nhóm dữ liệu từ tài khoản, học vụ, kho tri thức, hội thoại đến ticket]

3.2. Thiết kế API và Backend

3.2.1. Cấu trúc thư mục dự án

Mã nguồn được tổ chức theo kiến trúc phân tầng tập trung vào các endpoint API thay vì mô hình MVC đầy đủ. `views/` chứa các trang PHP hiển thị cho người dùng; `api/` tiếp nhận request từ trình duyệt và trả kết quả; `core/` chứa các hàm kết nối, chuẩn hóa và truy vấn dùng chung; `database/` chứa schema, seed và công cụ kiểm tra; `js/` xử lý tương tác phía client; `css/` định dạng giao diện; `tts/` chạy service FastAPI tạo âm thanh. Cấu trúc này thể hiện rõ sự tách biệt giữa tầng hiển thị, tầng xử lý request và tầng logic/truy vấn, nhưng chưa phải MVC đầy đủ vì chưa có thư mục controller và model tách biệt.

Mô hình thiết kế hướng đối tượng mô tả các thực thể như `User`, `Student` và `Admin`; trong mã nguồn PHP thuần, trách nhiệm của các thực thể này được ánh xạ thông qua session, kiểm tra role, các hàm dùng chung và các endpoint tương ứng thay vì khởi tạo thành một hệ thống class hoàn chỉnh. Cách tổ chức theo endpoint giúp mã nguồn nhẹ, dễ cô lập lỗi và phù hợp với kiến trúc đơn khối của dự án. Các hàm truy vấn trong `core/faq\_helpers.php` sử dụng PDO cùng câu lệnh có tham số, qua đó hạn chế nguy cơ SQL Injection khi xử lý dữ liệu đầu vào.

```
do_an_udpm/
├── api/           endpoint chatbot, feedback, ticket, TTS, avatar
├── core/          hàm dùng chung và truy vấn database
├── css/           login.css, dashboard.css, admin.css
├── database/      schema, seed và công cụ kiểm tra
├── js/            chatbot.js, admin.js
├── output/        tài liệu và kết quả xuất
├── scripts/       script hỗ trợ quá trình làm việc
├── tmp/           dữ liệu trung gian
├── tts/           server.py và requirements.txt
├── uploads/       file người dùng tải lên
├── views/         login.php, dashboard.php, admin_dashboard.php
├── .env           cấu hình môi trường
├── .htaccess      đặt trang login làm DirectoryIndex
├── config.php     đọc .env và kết nối MySQL/Gemini
├── README.md      hướng dẫn cài đặt
├── database_full_export.sql
├── fix_db.sh
├── fix_users.sql
├── seed_student.sql
└── test.js
```

Thành phần	Vai trò triển khai
views/login.php	Form đăng nhập, kiểm tra tài khoản và điều hướng theo role
views/dashboard.php	Dashboard Sinh viên, chatbot, hồ sơ, thông báo và ticket
views/admin_dashboard.php	Tổng quan, tri thức, người dùng, ticket và lịch sử chat
api/chatbot.php	Phân loại câu hỏi, truy vấn dữ liệu, RAG và trả JSON
api/*.php	Feedback, báo lỗi, ticket, hội thoại, TTS và avatar
core/faq_helpers.php	PDO, chuẩn hóa tiếng Việt, user, chat session, RAG view và ticket
config.php	Đọc .env, kết nối MySQL và lấy khóa Gemini
js/chatbot.js	Gửi câu hỏi, lịch sử cục bộ, feedback và phát TTS
tts/server.py	FastAPI/gTTS tạo file MP3 tiếng Việt

Bảng 3.3. Vai trò của các thành phần chính trong kiến trúc backend

Các hàm dùng chung trong `core/faq_helpers.php` sử dụng PDO, bật chế độ báo lỗi bằng exception và truyền tham số riêng cho câu lệnh SQL. Cách này giúp các endpoint dùng chung một cơ chế kết nối và hạn chế việc ghép trực tiếp dữ liệu người dùng vào câu lệnh truy vấn. `core/new_logic.php` chỉ ghi nhận rằng logic cũ đã được chuyển sang `api/chatbot.php`, vì vậy endpoint này là điểm xử lý chính của module Bot.

Mặc dù phương án thiết kế ban đầu định hướng sử dụng JWT cho xác thực, khi cài đặt thực tế trên nền tảng PHP thuần, nhóm chuyển sang PHP Session kết hợp với `session_regenerate_id(true)` sau khi đăng nhập thành công. Cơ chế này đồng bộ hơn với kiến trúc đơn khối của dự án vì trạng thái phiên được quản lý tập trung trên máy chủ, thay vì lưu token ở phía client như Local Storage. `views/login.php` gọi `password_verify()` để kiểm tra mật khẩu, kiểm tra trạng thái tài khoản, tạo session ID mới sau xác thực và lưu các thông tin cần thiết vào `$_SESSION`. Các trang quản trị kiểm tra role `'admin'`, `'staff'` hoặc `'knowledge_reviewer'`; dashboard Sinh viên kiểm tra role `'student'`.

Đây là thay đổi ở tầng hiện thực, không làm thay đổi luồng nghiệp vụ đăng nhập và phân quyền. Việc thay đổi session ID sau xác thực giúp giảm nguy cơ Session Fixation; việc không đưa JWT vào Local Storage cũng giảm rủi ro token bị đọc bởi mã JavaScript độc hại trong trường hợp xảy ra XSS. Do hệ thống chưa có benchmark so sánh, báo cáo không kết luận PHP Session nhanh hơn JWT mà chỉ ghi nhận đây là lựa chọn phù hợp với cách triển khai PHP thuần hiện tại.

3.2.2. Thiết kế các Endpoint API cốt lõi

Các endpoint nghiệp vụ đặt trong thư mục `api/`. Phần lớn endpoint nhận và trả JSON; riêng `api/text_to_speech.php` trả dữ liệu âm thanh khi proxy TTS thành công.

Endpoint	Dữ liệu chính	Chức năng
api/chatbot.php	POST JSON: message, sessionUuid, mssv	Phân loại, truy vấn database, RAG và trả reply
api/chat_feedback.php	POST JSON: messageId, rating, reasonCode, comment	Ghi đánh giá Hữu ích/Báo lỗi
api/report_bot.php	POST JSON: question, answer, messageId	Ghi lỗi và tạo ticket kèm ngữ cảnh
api/create_ticket.php	POST JSON: subject, description	Tạo ticket và ticket message đầu tiên
api/get_student_chat.php	GET: id	Sinh viên xem ticket thuộc MSSV của mình
api/get_ticket_chat.php	GET: id	Lấy hội thoại theo quyền Admin hoặc Sinh viên
api/student_reply.php	POST form: ticket_id, message	Sinh viên trả lời ticket chưa đóng
api/text_to_speech.php	POST JSON: text	Proxy đến FastAPI và trả audio/mpeg
api/upload_avatar.php	POST multipart: avatar	Kiểm tra và lưu avatar của user

Bảng 3.4. Danh sách endpoint API cốt lõi

`api/chatbot.php` đọc request JSON và từ chối thông điệp rỗng. Sau khi kết nối database, endpoint lấy hồ sơ sinh viên từ session hoặc MSSV phù hợp, phân loại câu hỏi bằng `classifyQuestion()` và nhận diện học kỳ, năm học, khóa sinh viên hoặc phạm vi ngày bằng `detectEntities()`. Một `chat\_session` được tìm hoặc tạo bằng UUID; tin nhắn người dùng và câu trả lời của hệ thống được ghi vào `chat\_messages`.

Câu hỏi về điểm, lịch học, lịch thi, học phí hoặc hồ sơ được truy vấn trực tiếp từ các bảng nghiệp vụ. Cách làm này giữ dữ liệu cá nhân ở tầng database, không đưa dữ liệu này vào prompt của Gemini. Với câu hỏi kiến thức chung, endpoint truy vấn `v\_active\_verified\_knowledge`, dùng Full-Text Search trên bài tri thức và chunk, sau đó kết hợp độ phủ từ khóa, intent, metadata, freshness và priority để tính điểm. Ngưỡng và số chunk tối đa được đọc từ `system\_settings`.

Mỗi kết quả truy xuất được lưu trong `rag\_retrieval\_logs`. Nếu không có chunk đạt ngưỡng, hệ thống ghi câu hỏi vào `unanswered\_questions`, trả thông báo fallback và gửi tín hiệu `TICKET\_OFFER` để giao diện đề nghị sinh viên tạo ticket. Nếu có khóa Gemini, các đoạn tri thức đã chọn được đưa vào context của model `gemini-2.5-flash-lite`; nếu không có khóa hoặc Gemini gặp lỗi, hệ thống vẫn có thể trả trực tiếp nội dung đã kiểm duyệt từ database.

Feedback được gửi qua `api/chat\_feedback.php`. Khi sinh viên báo lỗi câu trả lời, `api/report\_bot.php` ghi nhận rating không chính xác và tạo ticket gồm câu hỏi, câu trả lời gốc cùng thông tin người gửi. `api/create\_ticket.php` tạo ticket chủ động, thêm tin nhắn đầu tiên vào `ticket\_messages`; `api/get\_ticket\_chat.php` và `api/student\_reply.php` kiểm tra quyền sở hữu ticket trước khi trả hoặc ghi dữ liệu.

Module TTS được tách khỏi PHP. `tts/server.py` cung cấp `GET /health` và `POST /api/tts`, nhận văn bản tiếng Việt, tạo file MP3 bằng gTTS và trả `audio/mpeg`. `api/text\_to\_speech.php`

kiểm tra dữ liệu, gọi service tại `127.0.0.1:8001`, rồi chuyển tiếp audio về trình duyệt. `js/chatbot.js` nhận blob, phát bằng `Audio`, dừng âm thanh cũ khi cần và dùng `SpeechSynthesisUtterance` làm phương án dự phòng khi service Python không phản hồi.

[CHÈN HÌNH 3.2: Sơ đồ luồng xử lý câu hỏi từ giao diện đến PHP, MySQL, RAG và Gemini]

[CHÈN HÌNH 3.3: Sơ đồ luồng TTS từ chatbot.js đến PHP proxy và FastAPI/gTTS]

3.3. Thiết kế Frontend và Giao diện người dùng (UI/UX)

Giao diện được xây dựng bằng PHP, HTML, CSS và JavaScript thuần. `login.css`, `dashboard.css` và `admin.css` tách phần trình bày theo từng khu vực. Các thao tác tra cứu được đặt trong dashboard; chatbot và ticket mở dưới dạng cửa sổ tương tác để người dùng giữ được ngữ cảnh đang làm việc.

3.3.1. Sơ đồ điều hướng (Navigation Map)

Đăng nhập

- Sinh viên → Dashboard Sinh viên: tra cứu, thông báo, lịch học, chatbot và ticket
- Admin/Staff/Reviewer → Admin Dashboard: tổng quan, ticket, tri thức, sinh viên và lịch sử chat

`.htaccess` đặt `views/login.php` làm trang mặc định. Sau khi đăng nhập thành công, role `student` được chuyển đến `dashboard.php`; các role quản trị được chuyển đến `admin\_dashboard.php`. Trong Admin Dashboard, tab được xác định qua tham số `tab`, gồm `dashboard`, `tickets`, `faq`, `users` và `logs`. Các form quản trị gửi POST về chính `admin\_dashboard.php`, sau đó chuyển lại tab tương ứng.

[CHÈN HÌNH 3.4: Sơ đồ điều hướng tổng thể của hệ thống]

3.3.2. Mockup các màn hình chính

Màn hình đăng nhập chia thành khu vực tin tức và khu vực biểu mẫu. Người dùng nhập tài khoản, mật khẩu và có thể bật/tắt hiển thị mật khẩu. Khi thông tin không hợp lệ, thông báo được hiển thị ngay trên trang. Sau khi xác thực thành công, hệ thống điều hướng theo role tài khoản.

[CHÈN HÌNH 3.5: Giao diện màn hình đăng nhập]

Dashboard Sinh viên trình bày thông tin cá nhân, thẻ thống kê, thông báo, lịch học và các liên kết tra cứu nhanh. Avatar được tải qua `api/upload\_avatar.php`; các cửa sổ lịch, thông báo và ticket được mở ngay trong dashboard. Những nội dung như điểm, lịch học, lịch thi và học phí được lấy từ database thông qua luồng chatbot hoặc các vùng dữ liệu tương ứng.

[CHÈN HÌNH 3.6: Giao diện Dashboard Sinh viên]

Cửa sổ Chatbot gồm vùng hội thoại, ô nhập, nút gửi và các gợi ý. Mỗi câu trả lời có các thao tác nghe, sao chép, đánh giá hữu ích, báo lỗi và gửi yêu cầu hỗ trợ. Khi hệ thống trả tín hiệu tạo ticket, JavaScript hiển thị nội dung xác nhận trước khi gửi. Khi người dùng nghe câu trả lời mới, audio đang phát trước đó được dừng để tránh chồng tiếng.

[CHÈN HÌNH 3.7: Cửa sổ Chatbot với câu hỏi, câu trả lời và các nút thao tác]

Màn hình ticket hỗ trợ cho phép sinh viên xem nội dung trao đổi và gửi phản hồi khi ticket chưa đóng. Admin xem ticket theo trạng thái, mở cửa sổ trao đổi, gửi phản hồi và đóng ticket. Nội dung từng lượt trao đổi nằm trong `ticket\_messages`, còn trạng thái và thông tin chính nằm trong `tickets`.

[CHÈN HÌNH 3.8: Cửa sổ trao đổi ticket phía Sinh viên và phía Admin]

Admin Dashboard gồm các khu vực Tổng quan, Hỗ trợ Ticket, Kiểm duyệt tri thức, Quản lý Sinh viên và Lịch sử Chat. Khu vực tri thức cho phép thêm nguồn, thêm bài, sửa nội dung và thay đổi trạng thái kiểm duyệt. Khu vực Lịch sử Chat cho phép xem phiên hội thoại, trạng thái câu trả lời, feedback và ticket liên quan, tạo thành một luồng quản trị khép kín từ dữ liệu người dùng đến việc cập nhật kho tri thức.

[CHÈN HÌNH 3.9: Admin Dashboard tại khu vực Tổng quan]

[CHÈN HÌNH 3.10: Khu vực Kiểm duyệt tri thức]

Các thao tác phản hồi được đặt gần nội dung cần đánh giá để giảm số bước thao tác. Dữ liệu hiển thị từ PHP được xử lý bằng `htmlspecialchars()`; phía JavaScript có `escapeHTML()` và các hàm làm sạch nội dung trước khi đưa vào DOM. Cách xử lý này giúp giao diện hiển thị đúng nội dung tiếng Việt và hạn chế việc dữ liệu nhập vào được diễn giải như mã HTML hoặc JavaScript.

Thiết kế cơ sở dữ liệu, API và giao diện tạo thành một chuỗi xử lý thống nhất: dữ liệu được lưu theo nhóm nghiệp vụ, backend kiểm tra quyền và chọn đúng nhánh xử lý, còn giao diện cung cấp phản hồi trực tiếp cho người dùng. Các phần mở rộng như kho tri thức có kiểm duyệt, nhật ký truy xuất, feedback và ticket được tích hợp vào cùng luồng thay vì tồn tại như các chức năng tách rời.